



Rainbow Pixel

Documento de escopo da versão 1.0
Fonte da verdade do que a 1.0 entrega e do que fica preparado

PROJETO	Jogo de tiro tático 2D online, no navegador, com parede destrutível
VERSÃO DO DOCUMENTO	1.1 · 12/11/2026
RESPONSÁVEL	Pedro
STATUS	Aprovado para desenvolvimento

Como usar este documento

Este documento é a **fonte da verdade** da versão 1.0 do Rainbow Pixel. O que está aqui entra na 1.0. O que não está aqui **não entra**, mesmo que pareça pequeno. Ideia nova vai para a lista da 2.0 ou da 3.0 (seção 13), e não para o código da 1.0.

Números de balanceamento (dano, tempo, raio de som etc.) são **valores iniciais**. Eles ficam centralizados em `packages/shared/config` e podem ser ajustados durante os testes sem mudar este documento. Regras, escopo e arquitetura só mudam com uma nova versão do documento.

Seção	Conteúdo
1	Visão geral e pilares
2	Escopo da 1.0 em uma página
3	Regras da partida (Partida rápida, modo Bomba)
4	Recruta, movimento e combate
5	Arsenal (5 primárias + 3 secundárias)
6	Gadgets, drones e câmeras
7	Mapa, destruição e âncoras para gadgets futuros
8	Visão, som e informação
9	Conta e progressão
10	Arquitetura técnica e infraestrutura
11	Modelo de dados
12	Marcos internos e definição de pronto
13	Fora do escopo (2.0 e 3.0)
14	Riscos
15	Decisões assumidas por padrão
16	Bots
17	Ferramentas de teste (somente desenvolvimento)

Histórico de versões

Versão	Data	Mudança
1.0	12/11/2026	Primeira versão aprovada.
1.1	12/11/2026	Bots evolutivos desde a 1.0 (seção 16) e ferramentas de teste para desenvolver sozinho (seção 17). Ajustes nas seções 2, 7, 9, 10, 11, 12, 13, 14 e 15.

1 Visão geral

Rainbow Pixel é um jogo de tiro tático 2D, visto de cima, jogado no navegador, em partidas online 2v2. Inspirado no Rainbow Six Siege, o foco é **destruição de parede**, **informação** (drones, câmeras, som) e jogo de time de atacante contra defensor. É um projeto pessoal, sem objetivo de monetização, para jogar com amigos e possivelmente deixar público depois.

2v2 jogadores por partida	**MD5** Partida rápida, modo Bomba	**8px** célula de destruição	**30 Hz** tick do servidor	**8** armas no catálogo	**1** mapa, 1 andar

Pilares do jogo

Pilar	Significa na prática
Toda parede conta	Parede macia quebra com tiro e carga de breach. Furo de bala abre linha de visão real. A defesa decide o que reforçar.
Informação ganha round	Drone, câmera, som posicional e minimapa. Quem sabe onde o inimigo está vence.
Servidor manda	Toda a lógica roda no servidor. O cliente só desenha e envia input. Nada que o jogador não deveria ver chega ao navegador.
Preparado para crescer	O mapa já tem âncoras de porta, janela e chão. Modos e operadores já existem no catálogo, desativados.

2 Escopo da 1.0 em uma página

Área	Na 1.0 (jogável)	Já preparado para depois
Partida	2v2, modo Bomba, MD5, preparação 30s + ação 2min	—
Modos	Partida rápida	Sem colocação (2.0) e Ranqueada (3.0) cadastrados e desativados
Personagem	Recruta (atacante e defensor)	Operadores com habilidade e arsenal próprio (2.0)
Armas	5 primárias + 3 secundárias, nomes reais. Recruta escolhe 1 de cada.	Arsenal restrito por operador
Gadgets atacante	Carga de breach, drone	—
Gadgets defensor	Reforço de parede (2 por time), barricada de porta e janela	Arame, escudo móvel, bloqueio de entrada (âncoras prontas)
Tiro	Hitscan com recuo e dispersão, sem headshot, fogo amigo 50%	—
Mapa	1 mapa no Tiled, 1 andar, células de 8px, parede reforçada indestrutível	floorId, alçapão, âncoras de porta, janela e chão
Visão	Cone de 100° para onde mira, fog of war no servidor	—
Som	Posicional estéreo + indicador visual de direção	—
Informação	Drones, câmeras fixas do mapa, minimapa simples, ping de mapa	—
Morte	Assiste o aliado e usa drones (atacante) ou câmeras (defensor)	—
Conta	Login e senha sem verificação de e-mail, stats, histórico, loadout salvo, nível e XP	Verificação de e-mail e recuperação de senha (3.0)
Comunicação	Ping de mapa no jogo; voz pelo Discord	—
Plataforma	Desktop, WASD + mouse	—
Visual	Minimalista geométrico	—
Servidor	Homologação na máquina do Pedro; produção em São Paulo	—
Bots	Host adiciona bots no lobby; Fácil, Médio e Difícil; comportamento até o nível 3 (seção 16); partida com bot não conta stats nem XP	Bot nível 4+ (gadgets de ataque, drone, flanco, jogo de time) e modo treino
Ferramentas de teste	Seed de contas, sessão por aba, simulador de lag, overlay de debug, cliente headless (seção 17), só em desenvolvimento	—

3 Regras da partida

3.1 Formato: Partida rápida

Regra	Valor
Jogadores	2v2 (sala de até 4 jogadores, entrada por código)
Formato	Melhor de 5: vence quem ganhar 3 rounds
Lados	Rounds 1 e 2 em um lado, rounds 3 e 4 trocados. Round 5 tem lados sorteados.
Fase de preparação	30s. Atacantes ficam no spawn e controlam drones. Defensores reforçam e barricam.
Fase de ação	2min (relógio pausa a contagem regressiva quando a bomba é plantada e passa a valer o tempo da bomba)
Fogo amigo	Ativo, com 50% do dano
Reconexão	Jogador que cai volta ao mesmo round. O boneco fica parado e vulnerável por até 60s; depois disso conta como morto.

3.2 Modo Bomba

- O mapa tem **um par de alvos** (A e B) em salas vizinhas na 1.0. O schema aceita vários pares, para mapas futuros.
- Um atacante recebe o **desarmador** no início da ação, de forma aleatória. Ele pode largar o desarmador (tecla G) para o aliado pegar. Se o portador morrer, o desarmador cai no chão e qualquer atacante pode pegar.
- **Plantar:** 7s, só dentro da zona de um dos alvos, interrompível (reinicia do zero).
- **Bomba plantada:** detona em 45s.
- **Desarmar:** 7s, interrompível (reinicia do zero).

Quem vence o round	Condição
Ataque	Elimina todos os defensores antes do fim do tempo, ou a bomba detona
Defesa	Elimina todos os atacantes antes do plant, o tempo acaba sem plant, ou desarma a bomba
Caso especial	Se todos os atacantes morrerem depois do plant, a defesa ainda precisa desarmar para vencer

3.3 Spawns

- **Atacantes** escolhem entre 3 spawns fora do prédio durante a preparação. Ficam 3s invulneráveis ao entrar na fase de ação.
- **Defensores** nascem dentro do prédio, perto dos alvos.
- **Anti spawn-kill:** defensor que sair do polígono `building_interior` durante a ação aparece no minimapa dos atacantes a cada 3s.

4 Recruta, movimento e combate

Na 1.0 só existe o **Recruta**, um operador genérico com acesso a todo o catálogo de armas. Tecnicamente ele já é um registro do catálogo de operadores (`id: "recruit"`), então a 2.0 só adiciona registros novos, com arsenal restrito e uma habilidade.

Atributo	Valor inicial
Vida	100 (sem regeneração e sem cura na 1.0)
Hitbox	Círculo de 12px de raio, dano igual em qualquer ponto (sem headshot)
Andar	140 px/s, som de passo médio
Andar lento (Shift)	80 px/s, silencioso
Correr (Ctrl)	200 px/s, som alto, não pode atirar nem mirar
Loadout	1 primária + 1 secundária, escolhidas separadamente para ataque e defesa
Gadgets	Fixos por lado na 1.0 (seção 6)

4.1 Tiro

- **Hitscan**: o tiro é um raio instantâneo calculado no servidor.
- **Dispersão**: aumenta ao andar, ao correr (bloqueado) e a cada tiro seguido; volta ao normal ao parar de atirar.
- **Recuo**: padrão por arma, aplicado ao ângulo da mira.
- **Queda de dano**: depois do alcance efetivo, cai linearmente até 60% no dobro do alcance.
- **Penetração**: o raio atravessa até N células de parede macia (de acordo com a arma). Cada célula atravessada reduz 25% do dano e causa dano na própria célula.
- **Compensação de lag**: o servidor guarda 300ms de histórico de posições e volta no tempo até 200ms para validar o acerto.

5 Arsenal

O Recruta escolhe **1 primária e 1 secundária** de todo o catálogo. Os nomes são de armas reais (veja o risco na seção 14). Os valores abaixo são o ponto de partida do balanceamento.

5.1 Primárias

Arma	Categoria	Dano	Cadência	Pente	Recarga	Alcance	Penetr.	Mobil.
M4A1	Fuzil de assalto	32	750 RPM	30	2,4s	480px	2 células	1,00
MP5	Submetralhadora	26	800 RPM	30	2,1s	320px	1 célula	1,05
M870	Escopeta	8×12	70 RPM	6	0,5s/cart.	160px	1 célula*	1,00
M249	Metralhadora leve	30	750 RPM	100	5,0s	520px	3 células	0,85
SR-25	Fuzil de precisão (DMR)	60	300 (semi)	20	2,8s	640px	3 células	0,90

*A escopeta atravessa pouco, mas cada chumbo danifica a célula. É a melhor arma para abrir parede macia sem carga de breach.

5.2 Secundárias

Arma	Categoria	Dano	Cadência	Pente	Recarga	Alcance	Penetr.	Mobil.
M9	Pistola	30	450 (semi)	15	1,8s	280px	1 célula	1,00
.357 Magnum	Revólver	55	180 (semi)	6	3,0s	320px	2 células	1,00
Skorpion vz. 61	Pistola-metralhadora	20	850 RPM	20	1,9s	220px	0 células	1,00

Por que o catálogo fica no código e não no banco
Armas, gadgets, operadores e modos são lidos pela simulação no servidor e no cliente a cada tick. Eles ficam em packages/shared/catalog como dados tipados, com o campo enabled. O banco guarda só os ids (no loadout e no histórico). Trocar um nome ou valor de arma é mudar um arquivo.

6 Gadgets, drones e câmeras

Na 1.0, os gadgets são **fixos por lado**: todo atacante tem os mesmos e todo defensor tem os mesmos. O modelo de dados já tem o slot de gadget, para os operadores da 2.0 terem gadgets próprios.

Gadget	Lado	Quantidade	Regras
Carga de breach	Ataque	2 por jogador	Colocada em parede macia ou barricada, detona 2s depois de acionada. Remove as células num raio de 24px (3 células) e causa 80 de dano até 48px, inclusive em aliados (50%). Não afeta parede reforçada.
Drone	Ataque	2 por jogador	180 px/s, cone de 90° e alcance de 400px, quebra com 1 tiro. Passa por barricada de porta (fresta inferior) e por buracos de 16px ou mais. Marca inimigo no minimapa do time por 5s ao mirar nele por 0,5s.
Reforço de parede	Defesa	2 por time (compartilhado)	Instalado em 4s, só em segmentos marcados como reforçáveis no mapa (reinforce_slots). Deixa o segmento inteiro indestrutível na 1.0.
Barricada	Defesa	2 por jogador	Instalada em 2s numa âncora de porta ou janela. Bloqueia passagem e visão. É feita de células de material barricada (30 de vida cada), então tiro faz furo e soco ou corpo abre.
Câmera fixa	Defesa	Definida pelo mapa	Cadastrada no Tiled, cone de 110° e alcance de 500px, quebra com 1 tiro. Usada por defensor vivo (tecla) ou morto. Marca inimigo como o drone.

7 Mapa, destruição e âncoras

A 1.0 tem **um mapa de um andar**, criado no **Tiled** (editor gratuito) e convertido para o formato do jogo por um importador. Tamanho de referência: 240×160 células de 8px (1920×1280px). Todo elemento tem `floorId` (0 na 1.0), para o mapa aceitar vários andares sem migração.

7.1 Materiais das células

Material	Vida	Destrutível na 1.0	Observação
floor	—	Não	Chão comum. Flag deployable indica onde gadgets de chão podem ficar (2.0).
wall_soft	100	Sim	Tiro danifica, breach remove.
wall_soft + reforço	—	Não	Segmento reforçável que recebeu reforço.
wall_hard	—	Não	Estrutural e externa. Nunca quebra (até que um hard breacher exista na 2.0).
barricade	30	Sim	Gerada pelo gadget sobre uma âncora de porta ou janela.
void	—	—	Fora do mapa.

7.2 Convenção de camadas no Tiled

Camada	Tipo	Conteúdo
structure	Tiles	Materiais das células (propriedade material em cada tile).
anchors	Objetos	Portas, janelas e alçapões (futuro). Cada um com id, kind, floorId, facing e supports.
reinforce_slots	Objetos	Retângulos dos segmentos de parede que podem receber reforço.
gameplay	Objetos	spawn_attack (3), spawn_defense, objective (A e B com pairId), building_interior, camera (posição, rotação, fov), no_deploy.
bot_hints	Objetos	Dicas para bots: waypoint (patrulha), hold_point (posição de defesa por alvo), cover (cobertura), reinforce_priority e barricade_priority (ordem do que a defesa bot reforça e barrica), plant_spot.

7.3 Âncoras: o terreno para a 2.0

Cada porta e janela é uma âncora com a lista do que ela aceita. Na 1.0 só a barricada usa isso, mas os gadgets futuros (bloqueio de entrada estilo Kapkan, escudo em porta, arame no chão) só precisam ler a lista.

```
{
  "id": "door_cozinha_01",
  "kind": "door",                 // door | window | hatch (hatch: futuro)
  "floorId": 0,
  "rect": { "x": 512, "y": 320, "w": 32, "h": 8 },
  "facing": "N",                  // N | S | E | W
  "supports": ["barricade", "entry_denial", "deployable_shield"]
}
```

Gadget (versão)	Precisa de	Lê de
Barricada (1.0)	Porta ou janela	anchors.supports contém barricade
Bloqueio de entrada estilo Kapkan (2.0)	Porta ou janela	anchors.supports contém entry_denial
Escudo móvel (2.0)	Chão ou porta	célula floor com deployable, ou anchors.supports contém deployable_shield
Arame farpado (2.0)	Chão	célula floor com deployable, fora de zonas no_deploy

Regra do importador
O importador valida o mapa com zod e falha o build se algo estiver errado: âncora de porta fora de um vão de parede, alvo sem par, menos de 3 spawns de ataque, câmera fora do prédio, id duplicado. Mapa inválido nunca chega ao servidor.

7.4 Destruição em rede

- O grid completo é enviado uma vez, ao entrar na partida, em formato binário.
- Depois disso, só vão **eventos**: `{ tick, cells: [índice, vidaNova][] }`. O cliente aplica e redesenha só o trecho afetado.
- No início de cada round o grid volta ao estado original do mapa.

8 Visão, som e informação

8.1 Visão

- Cone de **100°** para onde o jogador mira, com alcance de 900px, bloqueado por paredes e barricadas.
- Visão periférica de 48px em 360°, para não ficar cego com alguém colado nas costas.
- Furos na parede abrem linha de visão real: a visão é recalculada quando o grid muda.
- **Fog of war no servidor:** a posição de um inimigo só é enviada ao cliente se ele estiver visível para o jogador, para um aliado, para um drone ou câmera do time, ou se estiver marcado. Esconder no cliente não é aceito.

8.2 Som

Evento	Raio de audição
Passos correndo	480px
Passos andando	240px
Andar lento	0 (silencioso)
Tiro	1200px
Carga de breach	1600px
Célula de parede quebrando	800px
Plantar ou desarmar	400px
Drone se movendo	160px

- Som abafado: cada parede entre a fonte e quem ouve reduz o raio efetivo em 40%.
- Áudio posicional estéreo (Web Audio API): volume pela distância e balanço esquerda/direita pela direção.
- Indicador visual: um arco na borda da tela mostra a direção e a intensidade do som.
- **Regra de segurança:** o servidor só envia um evento de som para quem está dentro do raio efetivo. Caso contrário, o indicador vira radar para quem abrir o DevTools.

8.3 Minimapa e ping

- Minimapa simples: planta do andar, alvos, aliados, marcações de drone e câmera (5s) e pings. A destruição não aparece no minimapa na 1.0.
- Ping de mapa (tecla Z): marca um ponto para o time por 4s, com limite de 1 a cada 1,5s.
- Voz fica no Discord. Não existe chat de voz no jogo.

8.4 Depois de morrer

- Atacante morto assiste o aliado e pode controlar os próprios drones que ainda estiverem inteiros.
- Defensor morto assiste o aliado e pode usar as câmeras fixas que ainda estiverem inteiras.
- Espectador recebe só o que o time vê (a mesma regra de fog of war).

9 Conta e progressão

Item	Na 1.0
Login	E-mail e senha via Supabase Auth, com confirmação de e-mail desligada. Nick único, de 3 a 16 caracteres.
Recuperação de senha	Não existe na 1.0. Reset manual pelo painel do Supabase.
Estatísticas	Abates, mortes, vitórias, derrotas, rounds jogados e vencidos, disparos, acertos (precisão), plants e desarmes.
Histórico	Partidas com data, placar, lado inicial, abates, mortes, XP ganho e duração.
Loadout salvo	Primária e secundária salvas separadamente para ataque e defesa (por operador; na 1.0, só recruit).
Nível e XP	Progressão simples por partida, sem recompensas desbloqueáveis na 1.0.
Partidas com bot	Não contam estatísticas nem XP. Aparecem no histórico marcadas como "com bots".

9.1 Tabela de XP inicial

Ação	XP
Participar da partida até o fim	+100
Round vencido	+50
Abate	+25
Plantar a bomba	+50
Desarmar a bomba	+75
Vencer a partida	+200
XP para passar do nível n para n+1	$1000 + 250 \times (n - 1)$

Quem abandona a partida não recebe XP. Stats e histórico são gravados em uma única transação no fim da partida.

10 Arquitetura técnica e infraestrutura

Regra de ouro: **o jogo roda no servidor**. A simulação é TypeScript puro, sem dependência de navegador, compartilhada entre servidor (verdade) e cliente (previsão do próprio movimento).

```
rainbow-pixel/                  (monorepo, pnpm workspaces)
├── packages/
│   ├── shared/                 simulação pura em TS
│   │   ├── sim/                movimento, colisão, raycast DDA, tiro, destruição, visão, som
│   │   ├── catalog/            armas, gadgets, operadores, modos (com enabled)
│   │   ├── config/             números de balanceamento
│   │   └── protocol/           tipos das mensagens cliente <-> servidor
│   └── maps/                   mapas exportados do Tiled + importador validado com zod
├── apps/
│   ├── server/                 Node + Colyseus + Express (API REST) + Prisma
│   ├── client/                 Vite + TS + PixiJS (jogo) + React (menus, lobby, perfil)
│   └── bots/                   cérebro dos bots em TS puro (percepção, memória, decisão)
└── tools/
    ├── map-import/             CLI: Tiled JSON -> MapDefinition
    └── headless-client/        cliente sem tela para testes (seção 17)
```

10.1 Netcode

Item	Valor ou técnica
Tick da simulação no servidor	30 Hz
Envio de estado aos clientes	20 Hz, filtrado por cliente (fog of war)
Input do cliente	30 Hz, numerado (seq) para reconciliação
Movimento próprio	Previsão no cliente + reconciliação com o estado do servidor
Outros jogadores	Interpolação com 100ms de atraso
Validação de tiro	Compensação de lag com histórico de 300ms (volta até 200ms)
Transporte	WebSocket via Colyseus. UDP (WebRTC ou WebTransport) fora do escopo.
Anti-cheat básico	Servidor autoritativo, limite de cadência e velocidade, fog e som filtrados no servidor

10.2 Infraestrutura

Peça	Homologação	Produção
Cliente (estático)	Vite na máquina do Pedro	Vercel
Servidor de jogo + API	Máquina do Pedro, exposta por Cloudflare Tunnel ou ngrok (ambos aceitam WebSocket)	Serviço em São Paulo (Fly.io região gru ou VPS nacional; confirmar regiões e planos atuais)
Banco	Supabase (projeto de homologação)	Supabase, região São Paulo
Autenticação	Supabase Auth	Supabase Auth. O servidor de jogo valida o JWT ao entrar na sala.

Regras de infraestrutura

1. Vercel não hospeda o servidor de jogo: serverless não segura WebSocket longo nem estado em memória.
2. O servidor de jogo nunca grava no banco durante a partida. Ele grava uma vez, no fim, em transação.
3. Em homologação, o ping dos amigos depende do upload da internet do Pedro. Teste de lag vale de verdade só em produção.
4. O CI aplica as migrations do Prisma de verdade. Pipeline verde sem migration aplicada é considerado falha.

11 Modelo de dados

O banco guarda só conta e resultado. Catálogo e mapas ficam no código (seção 5). O `Profile.id` é o mesmo id do usuário no Supabase Auth.

```
enum Side { ATTACK  DEFENSE }
enum RoundEndReason { ELIMINATION  BOMB_DETONATED  BOMB_DEFUSED  TIME_EXPIRED }

model Profile {
  id        String   @id @db.Uuid          // = auth.users.id
  nickname  String   @unique @db.VarChar(16)
  level     Int      @default(1)
  xp        Int      @default(0)
  createdAt DateTime @default(now())
  stats     PlayerStats?
  loadouts  Loadout[]
  matches   MatchPlayer[]
}

model PlayerStats {
  profileId    String @id @db.Uuid
  profile      Profile @relation(fields: [profileId], references: [id])
  kills        Int @default(0)
  deaths       Int @default(0)
  wins         Int @default(0)
  losses       Int @default(0)
  roundsPlayed Int @default(0)
  roundsWon    Int @default(0)
  shotsFired   Int @default(0)
  shotsHit     Int @default(0)
  plants       Int @default(0)
  defuses      Int @default(0)
}

model Loadout {
  id          String  @id @default(uuid()) @db.Uuid
  profileId   String  @db.Uuid
  profile     Profile @relation(fields: [profileId], references: [id])
  operatorId  String  @default("recruit")   // id do catálogo
  side        Side
  primaryId   String                        // id do catálogo de armas
  secondaryId String
  @@unique([profileId, operatorId, side])
}

model Match {
  id         String   @id @default(uuid()) @db.Uuid
  modeId     String                         // "quick" | "unranked" | "ranked"
  mapId      String
  startedAt  DateTime
  endedAt    DateTime
  scoreTeamA Int
  scoreTeamB Int
  winnerTeam Int?                           // 0 | 1, null se abandonada
  botCount   Int      @default(0)           // > 0: partida "com bots", sem stats e sem XP
  players    MatchPlayer[]
  rounds     MatchRound[]
}

model MatchPlayer {
  matchId   String  @db.Uuid
  profileId String  @db.Uuid
  match     Match   @relation(fields: [matchId], references: [id])
  profile   Profile @relation(fields: [profileId], references: [id])
  team      Int
  startSide Side
  kills     Int
  deaths    Int
  xpGained  Int
  abandoned Boolean @default(false)
  @@id([matchId, profileId])
}
```

```
model MatchRound {
  matchId    String @db.Uuid
  match      Match  @relation(fields: [matchId], references: [id])
  number     Int
  attackTeam Int
  winnerTeam Int
  reason     RoundEndReason
  durationMs Int
  @@id([matchId, number])
}
```

11.1 Catálogo já cadastrado

Catálogo	Registros	Ativos na 1.0
Modos	quick (Partida rápida), unranked (Sem colocação), ranked (Ranqueada)	quick
Operadores	recruit	recruit
Armas	M4A1, MP5, M870, M249, SR-25, M9, .357 Magnum, Skorpion vz. 61	Todas
Gadgets	breach_charge, drone, reinforcement, barricade	Todos
Mapas	1 mapa (nome a definir)	1

12 Marcos internos e definição de pronto

A 1.0 é grande. Ela é dividida em marcos com um critério de aceite cada. Um marco só começa quando o anterior passou no critério. Desde a versão 1.1 deste documento, cada marco também entrega o nível de bot e as ferramentas de teste correspondentes (seções 16 e 17).

Marco	Entrega	Critério de aceite
0.1 Fundação	Monorepo, CI (lint, typecheck, testes, migrations), homologação no ar, login, nick, criar e entrar em sala por código. Bots no lobby (adicionar, remover, dificuldade, troca por humano), sem comportamento. Seed de contas e sessão por aba.	Dois amigos fazem login e entram na mesma sala pela internet. Sozinho, com 4 contas de teste em abas, o lobby fecha 2v2. Host completa a sala com bots.
0.2 Movimento	Mapa do Tiled importado com colisão, movimento com previsão, reconciliação e interpolação. Simulador de lag, overlay de debug, cliente headless que conecta e anda. Bot nível 0.	Com 150ms de ping simulado, o próprio boneco não treme e os outros andam suave. Bot nível 0 patrulha os waypoints.
0.3 Tiro	Hitscan, recuo, dispersão, dano, morte, compensação de lag, 2 armas. Bot nível 1.	Tiro que acerta na tela do atirador conta no servidor em 95% dos casos com 150ms de ping. Bot nível 1 atira só em quem vê.
0.4 Destruição	Grid de 8px, penetração, carga de breach, eventos de destruição, reforço e barricada. Bot nível 2 (navegação).	Breach abre buraco igual para os 4 jogadores; reforço impede breach. Bot recalcula caminho por um buraco novo.
0.5 Visão e som	Cone de visão, fog of war no servidor, som posicional, indicador, minimapa base. Bot nível 2 completo (investiga som).	Inspecionando a rede, nenhuma posição de inimigo fora da visão chega ao cliente. Teste automatizado com cliente headless.
0.6 Modo Bomba	Preparação e ação, spawns, desarmador, plant e defuse, MD5, troca de lado, fogo amigo, reconexão. Bot nível 3 (objetivo).	Partida completa de 5 rounds sem intervenção manual, inclusive 1 humano contra 3 bots.
0.7 Informação	Drones, câmeras fixas, marcação, ping de mapa, espectador pós-morte. Bot atira em drone e câmera visíveis.	Marcação do drone aparece só para o time certo.
0.8 Arsenal	5 primárias + 3 secundárias, tela de loadout, loadout salvo. Bot sorteia loadout.	Loadout escolhido persiste entre sessões, por lado.

Marco	Entrega	Critério de aceite
0.9 Conta	Stats, histórico, XP e nível, tela de perfil, gravação no fim da partida. Partida com bot não conta.	Os números do perfil batem com as partidas jogadas. Partida com bot aparece no histórico e não altera stats nem XP.
1.0 Lançamento	HUD final, efeitos (screen shake, partículas), sons, tutorial curto, deploy de produção em São Paulo. Balanceamento das 3 dificuldades.	4 pessoas reais jogam 10 partidas seguidas sem crash nem dessincronização.

12.1 A 1.0 está pronta quando

- Todos os itens da seção 2 marcados como 1.0 funcionam em produção.
- Os critérios de aceite de todos os marcos passaram.
- Nenhum dado que o jogador não deveria ver (posição, som, marcação) chega ao navegador dele nem ao cérebro de um bot.
- O CI aplica migrations e roda os testes da simulação compartilhada e dos bots.
- Ferramentas da seção 17 estão desligadas em produção.
- Este documento foi revisado e nada da seção 13 entrou na 1.0.

13 Fora do escopo

Item	Versão prevista	O que já está preparado na 1.0
Operadores com habilidade e arsenal próprio	2.0	Catálogo de operadores, operatorId no loadout
Modo Sem colocação	2.0	Registro unranked no catálogo, modeId na partida
Arame farpado, escudo móvel, bloqueio de entrada estilo Kapkan	2.0	Âncoras com supports, flag deployable, zonas no_deploy
Hard breacher (quebra parede reforçada)	2.0	Material separado para parede reforçada
Modo Ranqueada	3.0	Registro ranked no catálogo
Verificação de e-mail e recuperação de senha	3.0	Supabase Auth (é só ativar)
Vários andares e alçapões	Sem data	floorId em tudo, kind hatch nas âncoras
Mais mapas	Sem data	Importador e pipeline de mapas
Bot nível 4+: breach planejado, drone, flanco, jogo de time, comunicação entre bots	2.0	Cérebro em camadas (seção 16), bot_hints no mapa
Modo treino solo (sem sala, contra bots, com configurações)	2.0	Bots no lobby, flag de partida com bots
Voz no jogo, gamepad, celular, editor de mapas próprio, fila pública	Sem data	—

14 Riscos

Risco	Impacto	Mitigação
O nome Rainbow Pixel lembra a marca Rainbow Six (Ubisoft) num jogo do mesmo gênero	Alto se o jogo ficar público	Uso interno sem problema. Reavaliar o nome antes de qualquer divulgação pública.
Nomes de armas reais podem ser marcas registradas de fabricantes	Baixo a médio se público	Os nomes ficam no catálogo: trocar é mudar um arquivo.
Escopo grande para um projeto solo	Projeto abandonado no meio	Marcos com critério de aceite; nada da seção 13 entra antes da 1.0.
Netcode e compensação de lag mais difíceis que o previsto	Atraso nos marcos 0.2 e 0.3	Testes com ping simulado desde o 0.2; simulação compartilhada coberta por testes.
Wallhack e radar pelo DevTools	Jogo injusto	Fog of war e filtro de som no servidor (critério do marco 0.5).
Ping alto com servidor fora do Brasil	Hitscan injusto	Produção em São Paulo.
Login sem recuperação de senha	Jogador perde a conta	Reset manual pelo painel na 1.0; recuperação na 3.0.
Limites do plano gratuito do Supabase (projeto inativo pode ser pausado)	Banco fora do ar	Gravação só no fim da partida; conferir as regras do plano atual.
Custo de CPU da visão (raycast para jogadores, drones e câmeras)	Servidor lento	Recalcular visão só quando algo muda e usar cache por célula.
O bot virar um projeto paralelo e atrasar os marcos	1.0 nunca sai	Um nível de bot por marco (seção 16.3). Meta do bot na 1.0 é ser útil para testar e jogar sozinho, não ser bom.
Custo de CPU dos bots (pathfinding e percepção)	Tick estoura 33ms	Decisão a 10 Hz, A* em grade de 32px com cache, recalcular só quando o grid muda; no máximo 3 bots por sala.
Ferramenta de teste ligada em produção	Trapaça ou falha de segurança	Tudo atrás de DEV_TOOLS=true, recusado na inicialização se NODE_ENV=production.

15 Decisões assumidas por padrão

Estes pontos não foram decididos explicitamente no planejamento e foram definidos por padrão para fechar o documento. Valem como estão até uma revisão.

Decisão	Motivo
Gadgets fixos por lado na 1.0 (sem escolha de gadget pelo Recruta)	Com só 2 gadgets por lado, a escolha seria trivial; o slot existe no modelo para a 2.0.
Drones também são o recurso do atacante morto	Equivalente às câmeras fixas para o defensor morto.
Um par de alvos fixo no mapa	Um mapa só na 1.0; o schema aceita vários pares.
Bomba detona em 45s; plantar e desarmar em 7s	Valores próximos do gênero, ajustáveis em config.
Round 5 com lados sorteados	O MD5 precisa de um critério para o round decisivo.
Defensor fora do prédio aparece no minimapa a cada 3s	Anti spawn-kill simples.
React para menus e PixiJS para o jogo	React já é a stack do Pedro; Pixi só desenha a partida.
Valores de armas, gadgets, som e XP	Ponto de partida do balanceamento, ajustável em config.
Bots rodam dentro do processo do servidor de jogo, no tick	Sem rede no meio, sem custo extra de infraestrutura.
Humano que entra em sala cheia de bots substitui um bot	Evita o host precisar remover bot manualmente para o amigo entrar.
Valores das dificuldades dos bots	Ponto de partida, ajustável em config.

16 Bots

O bot começa burro e evolui um nível por marco. Na 1.0 ele serve para duas coisas: **testar o jogo sozinho** e **completar a sala** quando faltam amigos. Ser um adversário bom não é meta da 1.0.

16.1 Princípios

- **Bot é um jogador sem teclado.** Ele recebe a mesma visão filtrada que o servidor mandaria para um cliente (PlayerView) e devolve o mesmo comando de input (InputCommand), que passa pelas mesmas validações de velocidade e cadência.
- **Bot nunca trapaceia.** Não lê o estado completo da partida. Se o fog of war vazar algo, o bot também vai "saber", então o bot serve de teste do fog of war.
- **Bot roda no servidor**, dentro do processo de jogo, decidindo a 10 Hz e repetindo o último comando nos ticks intermediários.
- **Bot nunca grava no banco.** Partida com pelo menos um bot tem botCount maior que zero e não altera stats nem XP de ninguém.
- **O cérebro é TypeScript puro** em packages/bots, sem dependência de Colyseus, para ser testado com cenários unitários e reutilizado pelo cliente headless.

16.2 Regras no lobby

- O host adiciona bot em vaga vazia escolhendo o time e a dificuldade, e pode remover a qualquer momento antes da partida.
- Bot está sempre pronto. A sala precisa de pelo menos 1 humano; no máximo 3 bots.
- Humano que entra por código numa sala cheia substitui o bot adicionado por último.
- Bot aparece com a tag BOT e a dificuldade no nome (ex.: BOT Médio 2). O loadout do bot é sorteado a cada round.

16.3 Níveis de comportamento

Nível	Marco	Comportamento
0 Boneco	0.2	Fica parado ou patrulha em linha reta entre waypoints de bot_hints. Serve de alvo.
1 Atirador	0.3	Ao ver um inimigo, gira a mira com velocidade limitada, espera o tempo de reação e atira com erro de mira. Recarrega quando o pente acaba. Sem inimigo, patrulha.
2 Navegador	0.4 e 0.5	Pathfinding A* numa grade de navegação de 32px gerada do grid de 8px e atualizada quando células são destruídas (buraco de 16px ou mais vira passagem). Lembra a última posição vista do inimigo e investiga sons ouvidos.
3 Objetivo	0.6 a 0.8	Máquina de estados por lado. Atacante: escolhe um alvo, pega o desarmador, vai até um plant_spot, planta e segura o plant. Defensor: na preparação reforça e barrica pela ordem de reinforce_priority e barricade_priority, segura um hold_point do alvo e vai desarmar ao ouvir o plant. Atira em drone e câmera inimigos visíveis.
4+ Tático	2.0	Fora da 1.0: carga de breach planejada, uso de drone, flanco, jogo de time e comunicação entre bots.

16.4 Dificuldades

Parâmetro	Fácil	Médio	Difícil
Tempo de reação ao ver inimigo	700ms	450ms	250ms
Erro de mira	±12°	±7°	±3°
Velocidade de giro da mira	180°/s	300°/s	480°/s
Rajada máxima antes de corrigir	3 tiros	6 tiros	Controle contínuo
Atenção a som	Só tiro e explosão	Todos os sons	Todos, com prioridade
Memória da última posição vista	3s	6s	10s

Mesmo cérebro para as três dificuldades: só mudam os parâmetros, em packages/shared/src/config.

16.5 Arquitetura do cérebro

```
packages/bots/src/
  BotBrain.ts        interface: decide(view: PlayerView, memory: BotMemory, dt) -> InputCommand
  perception.ts      extrai inimigos visíveis, sons e objetivo da PlayerView
  memory.ts          última posição vista, sons recentes, alvo atual (com tempo de expiração)
  navigation/        grade de 32px, A* com cache, invalidação por evento de destruição
  aim.ts             reação, giro limitado e erro de mira conforme a dificuldade
  levels/            level0.ts ... level3.ts (cada nível usa o anterior como base)
  scenarios/         testes: situação montada -> comando esperado
```

17 Ferramentas de teste (somente desenvolvimento)

Para desenvolver sozinho. Tudo fica atrás de `DEV_TOOLS=true`; o servidor se recusa a iniciar com essa flag se `NODE_ENV=production`.

Ferramenta	Marco	O que faz
Seed de contas	0.1	Script que cria 4 contas de teste com perfil (dev1 a dev4) usando a chave secreta do Supabase, que fica só no servidor.
Sessão por aba	0.1	Em desenvolvimento, a sessão do Supabase fica no sessionStorage: cada aba pode estar logada numa conta diferente.
Simulador de lag	0.2	Atraso, variação e perda de pacotes configuráveis por cliente, aplicados no servidor. O throttling do DevTools não é usado porque não é confiável para WebSocket.
Overlay de debug	0.2	Tick, ping, posição do servidor desenhada como fantasma sobre a posição prevista, contador de reconciliações.
Cliente headless	0.2 e 0.5	Script em Node que entra na sala como jogador, executa um roteiro de inputs e verifica o que recebeu (por exemplo, que não recebeu posição de inimigo atrás da parede).
Início forçado	0.6	Host inicia a partida sem todos prontos e com menos de 4 jogadores.
Comandos de debug	0.3 a 0.7	Teletransporte, vida infinita, mostrar a grade de navegação dos bots e o cone de visão de todos.